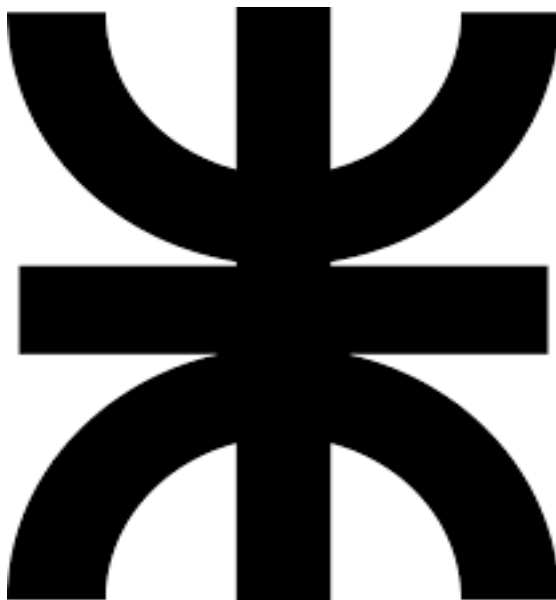


UNIDAD 1 - ACTIVIDAD 3

Laboratorio: Instalación y validación de un SGBD Relacional



UNIVERSIDAD TECNOLÓGICA NACIONAL
FACULTAD REGIONAL RESISTENCIA

Materia: Sistema de Gestión de Base de Datos.

Comisión: K 3.1 **Año:** 2026

Grupo: 6

Integrantes:

- Basualdo Alvarez, Facundo F.
- Godoy, Marianela Yasmin.
- Pare, Collante Agustina.
- Sanchez, Maria.
- Sanchez, Romina.
- Toledo, Francisco

Docentes:

- Prof. Andrés Fantin.
- JTP. Fernández, Juan Carlos.
- Aux. Edgar Nahuel Dominguez.

Índice

1. Identificación.....	4
2. Ambiente utilizado.....	4
3. Documentación oficial.....	5
4. Instalación.....	6
5. Evidencia de funcionamiento.....	7
6. Base de datos y objetos.....	9
7. Metadatos.....	16
8. Almacenamiento y configuración.....	18
9. Plan de ejecución.....	20
10. Persistencia.....	21
11. Mapa de arquitectura real.....	23
12. Incidente o decisión técnica.....	24
13. Uso de IA.....	25
14. Conclusión.....	26

1. Identificación

Ítem	Detalle
Grupo	N° 6 — Comisión K 3.1 (2026)
Integrantes	Basualdo Alvarez, Facundo F.; Godoy, Marianela Yasmin; Pare Collante, Agustina; Sanchez, María; Sanchez, Romina; Toledo, Francisco
Motor asignado	PostgreSQL (SGBD relacional de código abierto)
Versión instalada	18.4 — paquete Ubuntu 18.4-0ubuntu0.26.04.1
Cliente utilizado	psql 18.4 (CLI) y pgAdmin 4 (GUI, desde los equipos de cada integrante)
Fecha de realización	Del 18/08/2026 al 23/08/2026

El trabajo se organizó sobre una única instancia de PostgreSQL alojada en una máquina virtual con Ubuntu Server. Un integrante se ocupó del aprovisionamiento de la VM y de la instalación del motor; el resto del grupo instaló pgAdmin 4 en sus equipos y se conectó de forma remota a esa misma instancia, de modo que todas las pruebas se realizaron sobre la misma base de datos y con los mismos objetos.

2. Ambiente utilizado

Componente	Valor observado
Sistema operativo	Ubuntu Server 26.04 LTS (verificado con hostnamectl)
Kernel	Linux 7.0.0-29-generic
Virtualizador	Oracle VM VirtualBox (chassis: vm — Virtualization: oracle)
Arquitectura	x86_64
Memoria asignada	12 GB de RAM
Almacenamiento disponible	20 GB de disco virtual
Nombre del host	UbuntuServer
Red de trabajo	Red virtual Tailscale (VPN mesh) entre los equipos del grupo y la VM

```
fran@UbuntuServer:~$ hostnamectl
  Static hostname: UbuntuServer
        Icon name: computer-vm
        Chassis: vm
        Machine ID: a07a659cc897486784184545ba67a0a7
        Boot ID: c10f4c2aea4847879ddfd8348eb057d9
        Virtualization: oracle
Operating System: Ubuntu 26.04 LTS
        Kernel: Linux 7.0.0-29-generic
        Architecture: x86-64
        Hardware Vendor: innotek GmbH
        Hardware Model: VirtualBox
Hardware Version: 1.2
Firmware Version: VirtualBox
        Firmware Date: Fri 2006-12-01
        Firmware Age: 19y 8month 2w 3d
fran@UbuntuServer:~$
```

Salida de hostnamectl: versión del SO, kernel, arquitectura y virtualizador.

Observación sobre la versión del SO: la consigna plantea el laboratorio sobre Ubuntu 24.04. La imagen efectivamente utilizada en la VM reporta **Ubuntu 26.04 LTS**, lo que se confirma tanto en la salida de hostnamectl como en el sufijo del paquete instalado (18.4-0ubuntu0.26.04.1). Dejamos registrada la diferencia porque condiciona el procedimiento elegido: al tratarse de una versión de Ubuntu más reciente, el repositorio oficial de la distribución ya ofrece PostgreSQL 18.4, que es exactamente la versión indicada por la documentación del proyecto.

3. Documentación oficial

Ítem	Detalle
URL de instalación	https://www.postgresql.org/download/linux/ubuntu/
URL de referencia	https://www.postgresql.org/docs/18/index.html (documentación del servidor)
Procedimiento seleccionado	Instalación mediante paquetes APT desde el repositorio de la distribución
Fecha de consulta	18/08/2026
Versión indicada por la documentación	PostgreSQL 18.4 (última versión estable de la serie 18)
Versión finalmente instalada	PostgreSQL 18.4

Cómo interpretamos y aplicamos la guía oficial

La página oficial de descarga para Ubuntu propone dos caminos: (a) instalar el paquete que ya provee la distribución y (b) dar de alta el repositorio PGDG del proyecto para obtener la última versión disponible independientemente del ciclo de Ubuntu. Lo primero que hicimos fue verificar qué versión ofrecía nuestro sistema, porque el segundo camino sólo se justifica cuando el repositorio de la distribución quedó atrasado respecto de la versión requerida.

Al consultar el candidato disponible comprobamos que el repositorio de Ubuntu ya publicaba PostgreSQL 18.4, es decir, la misma versión que indicaba la documentación. Por ese motivo optamos por el procedimiento (a): es el camino más simple, mantiene el motor dentro del ciclo de

actualizaciones de seguridad de la distribución y evita agregar claves GPG y orígenes de paquetes adicionales. La documentación del proyecto se utilizó además como referencia para interpretar la estructura de directorios de Debian/Ubuntu (/etc/postgresql, /var/lib/postgresql, /var/log/postgresql), que no es la del código fuente original, y para entender el esquema de clústeres (pg_ctlcluster, unidades postgresql@18-main) propio del empaquetado.

Los blogs, foros y las herramientas de IA se utilizaron solamente como apoyo para interpretar mensajes de error y conceptos; la secuencia de instalación se tomó de la documentación oficial y del repositorio oficial de la distribución.

4. Instalación

Repositorio utilizado

Repositorio oficial de Ubuntu (componentes main / universe). No fue necesario incorporar el repositorio PGDG (apt.postgresql.org) porque la distribución ya ofrecía la versión 18.4 requerida.

Pasos principales y comandos relevantes

1. Sincronización del índice de paquetes con los servidores de Ubuntu:

```
sudo apt update
```

2. Instalación del servidor de base de datos y sus dependencias:

```
sudo apt install postgresql
```

3. Verificación del arranque automático del servicio luego de la instalación:

```
sudo systemctl status postgresql
sudo systemctl status postgresql@18-main
```

4. Verificación de la versión efectivamente instalada:

```
psql --version
# psql (PostgreSQL) 18.4 (Ubuntu 18.4-0ubuntu0.26.04.1)
```

Paquetes principales

Paquete	Función
postgresql	Metapaquete que instala la versión estable del motor para la distribución.
postgresql-18	Servidor propiamente dicho: proceso postgres, procesos auxiliares y catálogos.
postgresql-client-18	Cliente de línea de comandos psql y utilitarios (pg_dump, pg_isready, etc.).
postgresql-common	Infraestructura de clústeres de Debian/Ubuntu (pg_ctlcluster, pg_lsclusters).

Resultado final

La instalación finalizó sin errores. El paquete creó automáticamente el clúster 18/main, inicializó el directorio de datos, generó los archivos de configuración, dio de alta el usuario de sistema postgres y

dejó el servicio habilitado y escuchando en el puerto 5432. La versión instalada es PostgreSQL 18.4, coincidente con la indicada por la documentación oficial.

```
fran@UbuntuServer:~$ psql --version
psql (PostgreSQL) 18.4 (Ubuntu 18.4-0ubuntu0.26.04.1)
fran@UbuntuServer:~$
```

Verificación de la versión instalada del motor.

5. Evidencia de funcionamiento

Elemento	Resultado observado
Motor	PostgreSQL 18.4
Servicio (unidad envoltorio)	postgresql.service — PostgreSQL RDBMS
Servicio real del clúster	postgresql@18-main.service — PostgreSQL Cluster 18-main
Proceso principal	/usr/lib/postgresql/18/bin/postgres -D /var/lib/postgresql/18/main
PID del proceso principal	9850
Usuario del SO que ejecuta el motor	postgres
Usuarios del SO de los integrantes	facubas, fran, maria, agus, yas y romi
Puerto	5432 (TCP) y socket /var/run/postgresql
Cliente CLI	psql -h 127.0.0.1 -p 5432 -U postgres -d sgbd_lab

Servicio

En el empaquetado de Ubuntu, postgresql.service es una unidad "envoltorio" que no ejecuta el motor (su ExecStart es /bin/true, por eso aparece como active (exited)): su función es arrastrar el arranque y la detención de todos los clústeres configurados. El servicio que realmente mantiene el motor en ejecución es postgresql@18-main.service, que aparece como active (running). Esta distinción explica por qué la primera unidad muestra un PID que ya finalizó y la segunda muestra el PID del proceso servidor vivo.

```
facubas@UbuntuServer:~$ sudo systemctl status postgresql
• postgresql.service - PostgreSQL RDBMS
   Loaded: loaded (/usr/lib/systemd/system/postgresql.service; enabled; preset: enabled)
   Active: active (exited) since Tue 2026-08-18 21:05:49 UTC; 1h 28min ago
  Invocation: db9ad39697a24be9a38edd0f9e230ed3
    Process: 1240 ExecStart=/bin/true (code=exited, status=0/SUCCESS)
   Main PID: 1240 (code=exited, status=0/SUCCESS)
    Mem peak: 1.7M
       CPU: 8ms

Aug 18 21:05:49 UbuntuServer systemd[1]: Starting postgresql.service - PostgreSQL RDBMS...
Aug 18 21:05:49 UbuntuServer systemd[1]: Finished postgresql.service - PostgreSQL RDBMS.
facubas@UbuntuServer:~$
```

postgresql.service: unidad envoltorio, habilitada y ejecutada al inicio.

```

● postgresql@18-main.service - PostgreSQL Cluster 18-main
   Loaded: loaded (/usr/lib/systemd/system/postgresql@18-main.service; enabled-runtime)
   Active: active (running) since Sun 2026-08-23 22:31:43 UTC; 23min ago
     Process: 9845 ExecStart=/usr/bin/pg_ctlcluster --skip-systemctl-redirect 18-main start
    Main PID: 9850 (postgres)
      Tasks: 12 (limit: 6204)
     Memory: 45.9M (peak: 50.9M)
    CGroup: /system.slice/system-postgresql.slice/postgresql@18-main.service

```

postgresql@18-main.service: clúster activo (running), PID principal 9850, usuario postgres.

Proceso principal, PID y usuario del sistema operativo

El proceso principal (postmaster) es `/usr/lib/postgresql/18/bin/postgres`, ejecutado por el usuario de sistema `postgres` con PID 9850, apuntando al directorio de datos `/var/lib/postgresql/18/main` y al archivo de configuración `/etc/postgresql/18/main/postgresql.conf`. De él dependen los procesos auxiliares (checkpointer, walwriter, autovacuum) y un proceso backend por cada conexión.

```
ps -o pid,ppid,user,etime,command -C postgres
```

```

  PID  PPID  USER      ELAPSED  COMMAND
  9850    1 postgres   23:37 /usr/lib/postgresql/18/bin/postgres
          -D /var/lib/postgresql/18/main
          -c config_file=/etc/postgresql/18/main/postgresql.conf

```

Proceso principal: PID 9850, usuario postgres, PGDATA y archivo de configuración.

Puerto y conexión mediante cliente CLI

La disponibilidad del servicio se comprobó con `pg_isready`, que informa el socket y el puerto en escucha (5432) y el estado `accepting connections`. La conexión desde el cliente de línea de comandos se realizó con:

```
psql -h 127.0.0.1 -p 5432 -U postgres -d sgbd_lab
sudo -u postgres psql
```

```

Fran@UbuntuServer:~$ pg_isready
/var/run/postgresql:5432 - accepting connections
Fran@UbuntuServer:~$

```

pg_isready: el motor acepta conexiones en el puerto 5432.

```

Fran@UbuntuServer:~$ w
23:00:39 up 1:55, 4 users, load average: 0.00, 0.00, 0.00
USER      TTY      FROM            LOGIN@   IDLE   JCPU   PCPU   WHAT
Fran     pts/0    10.13.18.219    22:46    1.00s  0.06s  ?      w
Fran     10.13.18.219    22:21          0.00s  0.01s  sshd-session: fran [priv]
Facubas   10.0.2.2    21:21          0.00s  0.01s  sshd-session: facubas [priv]
Facubas   tty1      -              21:05    1:34  0.12s  0.01s  sudo -u postgres psql
Fran@UbuntuServer:~$

```

Sesiones abiertas sobre el servidor (comando w): conexiones SSH de los integrantes y sesión local de psql.

6. Base de datos y objetos

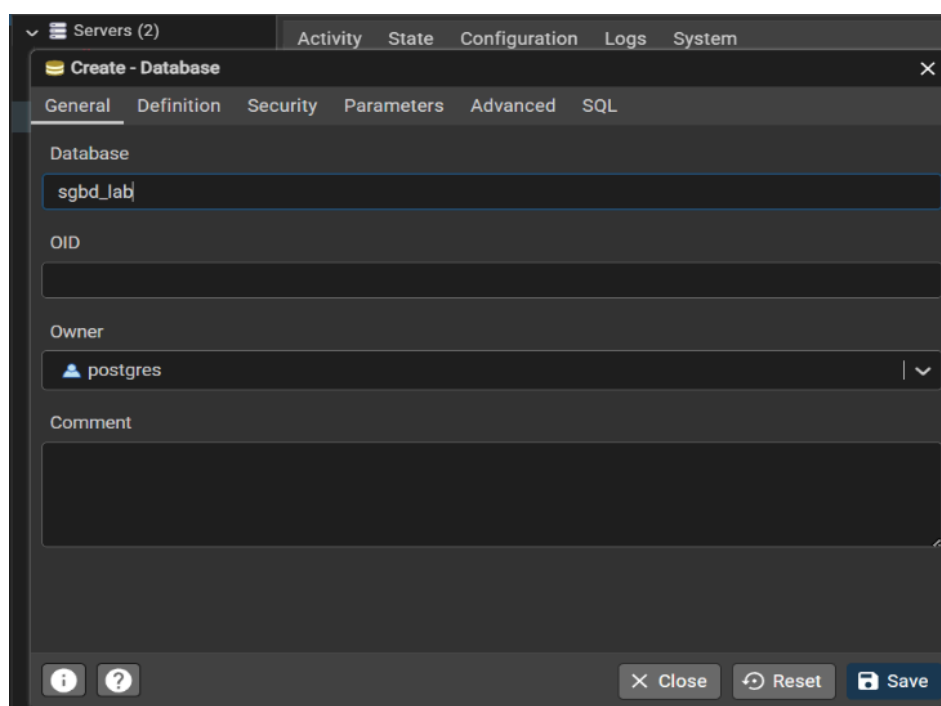
6.1. Creación de la base sgbd_lab

Desde pgAdmin 4 se hizo clic derecho sobre el nodo **Databases** y se eligió **Create** → **Database**, indicando el nombre `sgbd_lab` y dejando a `postgres` como propietario. Al guardar, el motor inicializó la estructura lógica de la nueva base. La sentencia equivalente en SQL es:

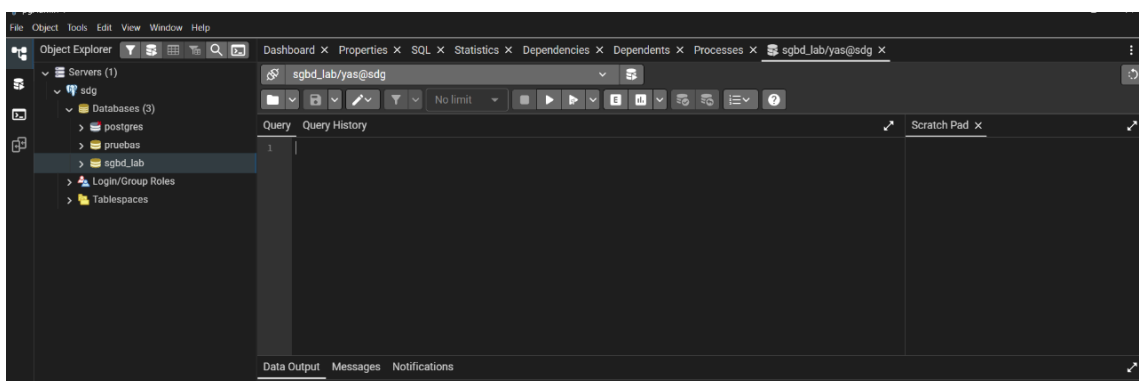
```
CREATE DATABASE sgbd_lab OWNER postgres;
```



Menú *Create* → *Database* en pgAdmin 4.



Diálogo de creación de la base `sgbd_lab` con propietario `postgres`.

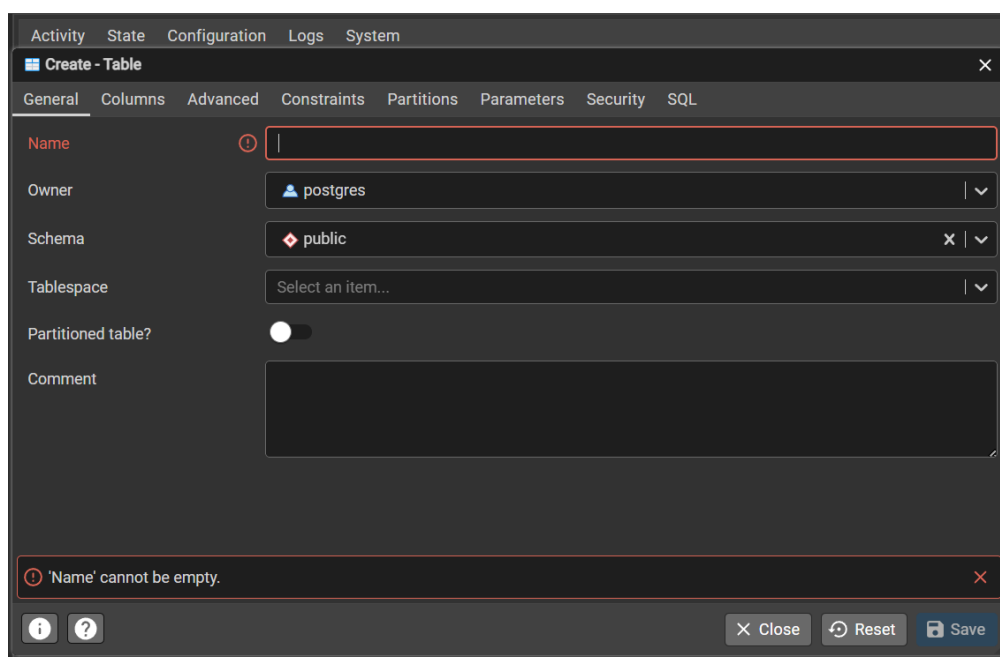


Query Tool abierto sobre `sgbd_lab`, listo para ejecutar DDL y DML.

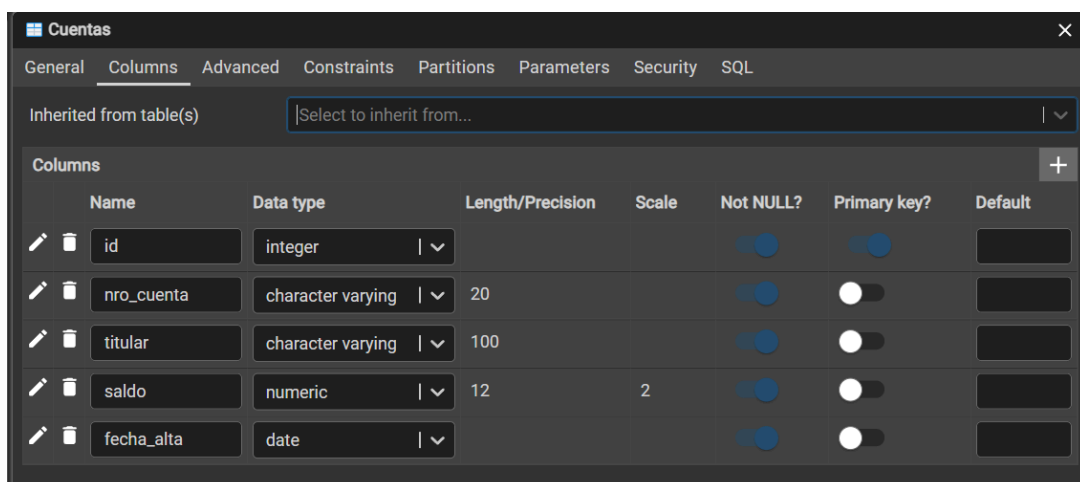
6.2. Creación de la tabla cuentas

La tabla se definió primero con el asistente gráfico Create - Table, dentro del esquema public, con los atributos solicitados por la consigna:

Campo	Tipo conceptual	Tipo en PostgreSQL	Restricciones
id	entero	integer (identity)	PRIMARY KEY, GENERATED ALWAYS AS IDENTITY
nro_cuenta	texto	character varying(20)	NOT NULL, UNIQUE
titular	texto	character varying(100)	NOT NULL
saldo	decimal	numeric(12,2)	NOT NULL
fecha_alta	fecha	date	NOT NULL



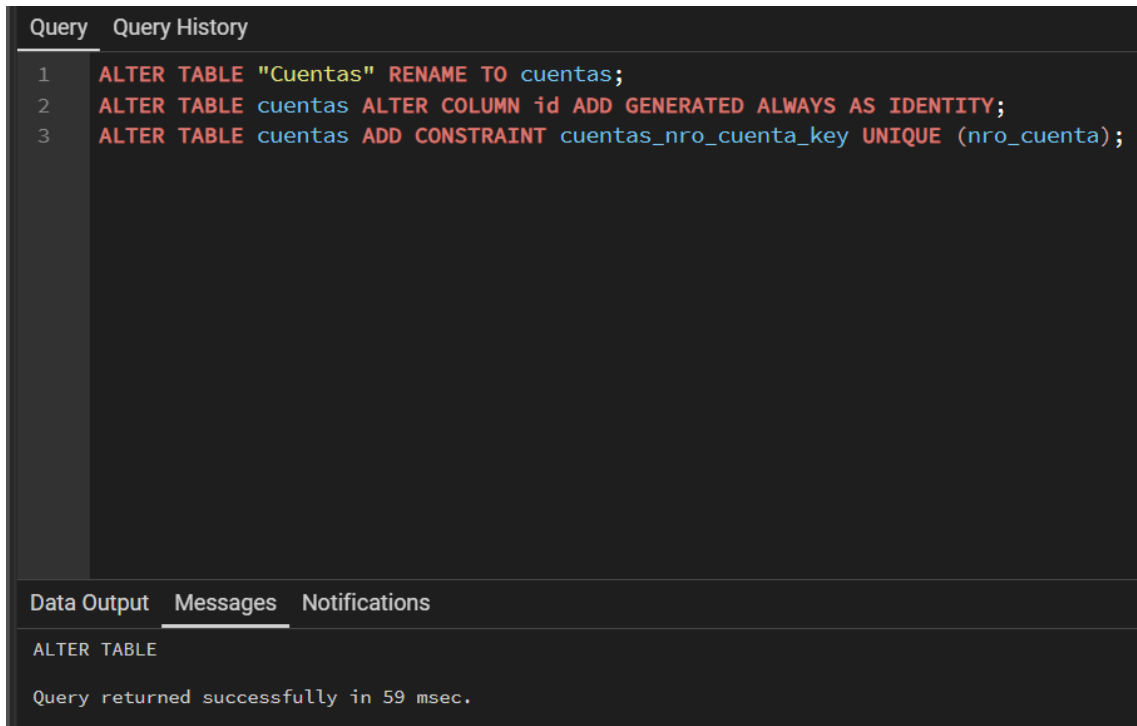
Pestaña General del diálogo Create - Table (esquema public, propietario postgres).



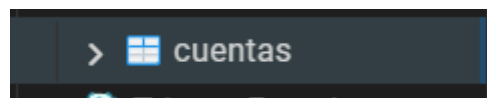
Pestaña Columns con los atributos y tipos definidos.

Incidencia: la tabla se creó inicialmente como "Cuentas" con mayúscula inicial. Como PostgreSQL conserva las mayúsculas cuando el identificador se crea entrecomillado, toda referencia posterior exigía escribirlo entre comillas dobles. Se normalizó el nombre a minúsculas y se completaron las restricciones con:

```
ALTER TABLE "Cuentas" RENAME TO cuentas;  
ALTER TABLE cuentas ALTER COLUMN id ADD GENERATED ALWAYS AS IDENTITY;  
ALTER TABLE cuentas ADD CONSTRAINT cuentas_nro_cuenta_key UNIQUE (nro_cuenta);
```



Normalización del nombre del objeto y alta de restricciones.



La tabla cuentas ya visible en el árbol de objetos de pgAdmin.

La definición resultante, expresada en SQL estándar, equivale a:

```
CREATE TABLE cuentas (  
    id            integer GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
    nro_cuenta    varchar(20)  NOT NULL UNIQUE,  
    titular       varchar(100) NOT NULL,  
    saldo         numeric(12,2) NOT NULL,  
    fecha_alta    date         NOT NULL  
);
```

6.3. Registros insertados

```
INSERT INTO cuentas (nro_cuenta, titular, saldo, fecha_alta) VALUES  
( '0001-CA', 'Facundo Basualdo', 15000.50, '2026-03-01'),  
( '0002-CC', 'Agustina Pere',   8200.00, '2026-03-05'),  
( '0003-CA', 'Francisco Toledo', 2350.75, '2026-04-10'),  
( '0004-CC', 'Maria Sancrez',   47800.20, '2026-05-22'),  
( '0005-CA', 'Marianela Godoy',  990.00, '2026-06-30');
```

```
Query  Query History
1  INSERT INTO cuentas (nro_cuenta, titular, saldo, fecha_alta) VALUES
2  ('0001-CA', 'Facundo Basualdo', 15000.50, '2026-03-01'),
3  ('0002-CC', 'Agustina Pere', 8200.00, '2026-03-05'),
4  ('0003-CA', 'Francisco Toledo', 2350.75, '2026-04-10'),
5  ('0004-CC', 'Maria Sancrez', 47800.20, '2026-05-22'),
6  ('0005-CA', 'Marianela Godoy', 990.00, '2026-06-30');

Data Output  Messages  Notifications
INSERT 0 5
Query returned successfully in 59 msec.
```

Carga inicial: INSERT 0 5, cinco registros incorporados.

6.4. Operaciones SELECT, INSERT, UPDATE y DELETE

SELECT

```
SELECT titular FROM cuentas;
```

Query Query History

```
1 SELECT titular from cuentas
```

Data Output Messages Notifications

	titular character varying (100) 🔒
1	Facundo Basualdo
2	Agustina Pere
3	Francisco Toledo
4	Maria Sancrez
5	Marianela Godoy

Consulta de los titulares cargados en la tabla.

INSERT

```
INSERT INTO cuentas (nro_cuenta, titular, saldo, fecha_alta)
VALUES ('0006-CA', 'Romina Sanchez', 90.00, '2026-06-30');
```

The screenshot shows a SQL IDE interface with a dark theme. The top bar has tabs for 'Query' and 'Query History'. The 'Query' tab is active, displaying a SQL query: `INSERT into cuentas ( nro_cuenta , saldo , fecha_alta ) values ('0006-CA', 'Romina Sanchez', 90.00, '2026-06-30');`. A tooltip with 'Execute script' and 'F5' is visible over the query. Below the query editor, there are tabs for 'Data Output', 'Messages', and 'Notifications'. The 'Messages' tab is active, showing the output: `INSERT 0 1` and `Query returned successfully in 45 msec.`

Alta de un sexto registro: INSERT 0 1.

UPDATE

```
UPDATE cuentas SET saldo = 12000 WHERE titular = 'Agustina Pere';
```

The screenshot shows a SQL IDE interface with a dark theme. The top bar has a toolbar with icons for file operations, query execution, and other functions. The 'Query' tab is active, displaying a SQL query: `UPDATE cuentas SET saldo = 12000 WHERE titular = 'Agustina Pere';`. Below the query editor, there are tabs for 'Data Output', 'Messages', and 'Notifications'. The 'Messages' tab is active, showing the output: `UPDATE 1` and `Query returned successfully in 30 msec.`

Modificación del saldo: UPDATE 1.

DELETE

```
DELETE FROM cuentas WHERE nro_cuenta = '0002-CC';
```

The screenshot shows a database query tool with two tabs: 'Query' and 'Query History'. The 'Query' tab is active, displaying a SQL query with line numbers 1 and 2. Line 1 is 'DELETE FROM cuentas' and line 2 is 'WHERE nro_cuenta = '0002-CC';'. Below the query, there are three tabs: 'Data Output', 'Messages', and 'Notifications'. The 'Messages' tab is active, showing the output 'DELETE 1' and a status message 'Query returned successfully in 52 msec.'

```
Query  Query History
1  DELETE FROM cuentas
2  WHERE nro_cuenta = '0002-CC';

Data Output  Messages  Notifications
DELETE 1
Query returned successfully in 52 msec.
```

Baja de un registro: DELETE 1.

COMMIT

El Query Tool de pgAdmin 4 trabaja por defecto en modo **autocommit**: cada sentencia se confirma automáticamente al finalizar, por lo que no fue necesario escribir COMMIT de forma explícita. Cuando se requiere control manual de la transacción, se desactiva el autocommit o se abre la transacción explícitamente con BEGIN ... COMMIT / ROLLBACK.

6.5. Índice creado

Sobre `nro_cuenta` la restricción UNIQUE ya generó automáticamente el índice `cuentas_nro_cuenta_key`, que es el que utiliza el optimizador para las búsquedas por número de cuenta. Adicionalmente, para el desafío del plan de ejecución se creó un índice sobre la tabla auxiliar `movimientos` y un índice trigram sobre `titular`:

```
-- Índice equivalente al generado por la restricción UNIQUE
CREATE UNIQUE INDEX cuentas_nro_cuenta_key ON public.cuentas USING btree (nro_cuenta);

-- Índices utilizados en la prueba de plan de ejecución
CREATE INDEX idx_mov_cuenta_id ON public.movimientos USING btree (cuenta_id);
CREATE INDEX idx_cuentas_titular_trgm ON public.cuentas USING gin (titular
gin_trgm_ops);
```

7. Metadatos

7.1. Columnas y tipos de datos

```
SELECT ordinal_position      AS pos,  
       column_name          AS columna,  
       data_type             AS tipo,  
       character_maximum_length AS largo,  
       numeric_precision     AS precision,  
       numeric_scale         AS escala,  
       is_nullable          AS acepta_null,  
       is_identity           AS es_identity  
FROM information_schema.columns  
WHERE table_schema = 'public'  
      AND table_name = 'cuentas'  
ORDER BY ordinal_position;
```

The screenshot shows a database query tool interface. At the top, there's a 'Query' tab with the SQL query. Below it, the 'Data Output' tab shows the results of the query as a table with 9 columns: pos, columna, tipo, largo, precision, escala, acepta_null, and es_identity. The results show 5 rows of data for the 'cuentas' table.

pos	columna	tipo	largo	precision	escala	acepta_null	es_identity
1	id	integer	[null]	32	0	NO	YES
2	nro_cuen...	character varying	20	[null]	[null]	NO	NO
3	titular	character varying	100	[null]	[null]	NO	NO
4	saldo	numeric	[null]	12	2	NO	NO
5	fecha_alta	date	[null]	[null]	[null]	NO	NO

Resultado de la consulta al diccionario de datos: cinco columnas con sus tipos.

Resultado obtenido:

Pos	Columna	Tipo	Largo / precisión	Acepta NULL	Identity
1	id	integer	32	NO	YES
2	nro_cuenta	character varying	20	NO	NO
3	titular	character varying	100	NO	NO
4	saldo	numeric	12, 2	NO	NO
5	fecha_alta	date	—	NO	NO

7.2. Esquema y propietario

```
SELECT n.nspname AS esquema,
       c.relname AS tabla,
       pg_get_userbyid(c.relowner) AS propietario,
       c.relkind AS tipo_objeto,
       c.reltuples::bigint AS filas_estimadas,
       pg_size_pretty(pg_total_relation_size(c.oid)) AS tamaño
FROM pg_class c
JOIN pg_namespace n ON n.oid = c.relnamespace
WHERE c.relname = 'cuentas'
      AND n.nspname = 'public';
```

The screenshot shows a database query interface with a dark theme. At the top, there are tabs for 'Query' and 'Query History'. The 'Query' tab is active, displaying the SQL query from the previous block. Below the query, there are tabs for 'Data Output', 'Messages', and 'Notifications'. The 'Data Output' tab is active, showing a table of results. The table has 7 columns: 'esquema', 'tabla', 'propietario', 'tipo_objeto', 'filas_estimadas', and 'tamaño'. The first row of data shows 'public' for the schema, 'cuent...' for the table, 'postgres' for the owner, 'r' for the object type, '-1' for the estimated rows, and '40 kB' for the size. The interface also includes a toolbar with icons for various actions like copy, paste, and execute, and a 'Showing' indicator.

	esquema name	tabla name	propietario name	tipo_objeto "char"	filas_estimadas bigint	tamaño text
1	public	cuent...	postgres	r	-1	40 kB

Esquema public, propietario postgres, relkind = r (tabla ordinaria), 40 kB de tamaño total.

7.3. Índices

```
SELECT indexname AS indice, indexdef AS definicion
FROM pg_indexes
WHERE schemaname = 'public'
      AND tablename = 'cuentas';
```

```

1 SELECT indexname AS indice, indexdef AS definicion
2 FROM pg_indexes
3 WHERE schemaname = 'public'
4 AND tablename = 'cuentas';

```

	indice name	definicion text
1	Cuentas_pkey	CREATE UNIQUE INDEX "Cuentas_pkey" ON public.cuentas USING btree (id)
2	cuentas_nro_cuenta_k...	CREATE UNIQUE INDEX cuentas_nro_cuenta_key ON public.cuentas USING btree (nro_cue...

Índices existentes sobre cuentas: clave primaria y clave única sobre nro_cuenta.

7.4. ¿Dónde almacena el motor la información sobre sus propias tablas?

PostgreSQL guarda la definición de sus objetos en el **catálogo del sistema**, un conjunto de tablas y vistas internas que el propio motor consulta y actualiza cada vez que se ejecuta una sentencia DDL o se planifica una consulta. Ese catálogo vive en el esquema `pg_catalog` y está presente en cada base de datos: `pg_class` registra tablas, índices y vistas; `pg_attribute` las columnas; `pg_namespace` los esquemas; `pg_index` los índices; `pg_authid` los roles y permisos.

Por encima de ese catálogo nativo, el motor expone `INFORMATION_SCHEMA`, un conjunto de vistas normalizado por el estándar SQL que presenta la misma información con nombres portables entre distintos SGBD. Ambos se consultan con sentencias `SELECT` comunes, lo que constituye una muestra concreta del principio de que el diccionario de datos es, él mismo, una base de datos relacional.

A nivel físico, esas tablas del catálogo se almacenan igual que cualquier otra: dentro del directorio `PGDATA/base/<OID>`, en archivos nombrados por el identificador de objeto correspondiente.

8. Almacenamiento y configuración

Elemento	Ubicación en el sistema de archivos
Directorio de datos (PGDATA)	/var/lib/postgresql/18/main
Archivo de configuración principal	/etc/postgresql/18/main/postgresql.conf
Autenticación de clientes	/etc/postgresql/18/main/pg_hba.conf
Mapado de usuarios	/etc/postgresql/18/main/pg_ident.conf

Elemento	Ubicación en el sistema de archivos
Logs del servidor	/var/log/postgresql/postgresql-18-main.log
Log transaccional (WAL)	/var/lib/postgresql/18/main/pg_wal
Datos de las bases	/var/lib/postgresql/18/main/base/<OID de la base>

8.1. Directorio de datos

```
ps aux | grep postgres | grep -E '\-D'
sudo -u postgres psql -c "SHOW data_directory;"
```

```
maria@UbuntuServer:~$ ps aux | grep postgres | grep -E '\-D'
postgres 1198 0.0 0.4 227580 33868 ?        Ss   21:05   0:00 /usr/lib/postgresql/18/bin/postgres -D
/var/lib/postgresql/18/main -c config_file=/etc/postgresql/18/main/postgresql.conf
```

El parámetro -D del proceso servidor revela el directorio de datos en uso.

8.2. Archivos de configuración

```
sudo -u postgres psql
postgres=# SHOW config_file;
```

```
fran@UbuntuServer:~$ sudo -u postgres psql
psql (18.4 (Ubuntu 18.4-0ubuntu0.26.04.1))
Type "help" for help.

postgres=# SHOW config_file;
          config_file
-----
/etc/postgresql/18/main/postgresql.conf
(1 row)
```

Ubicación del archivo postgresql.conf informada por el propio motor.

En el empaquetado de Debian/Ubuntu la configuración se separa de los datos: los archivos editables viven en /etc/postgresql/18/main/ y el motor los recibe por línea de comandos mediante -c config_file=..., mientras que PGDATA queda reservado exclusivamente para el almacenamiento.

8.3. Logs del servidor

Según la documentación oficial y la convención del paquete, los registros del servidor se escriben en /var/log/postgresql/, con un archivo por clúster (postgresql-18-main.log). Allí quedan los mensajes de arranque y detención, los errores de conexión y las sentencias que superan los umbrales configurados. Los eventos de la unidad de systemd, en cambio, se consultan con journalctl -u postgresql@18-main.

8.4. Log transaccional (WAL)

```
sudo -u postgres psql -t -P format=unaligned \
-c "SHOW data_directory;" | sed 's/$/\pg_wal/'
```

```
maria@UbuntuServer:~$ sudo -u postgres psql -t -P format=unaligned -
-c "SHOW data_directory;" | sed 's/$/\pg_wal/'
/var/lib/postgresql/18/main/pg_wal
```

Ubicación del log transaccional: /var/lib/postgresql/18/main/pg_wal.

8.5. ¿Qué elementos físicos representan a la base de datos creada?

Cada base de datos se materializa como un subdirectorio dentro de PGDATA/base/ cuyo nombre es el **OID** (identificador de objeto) de esa base. Dentro de ese subdirectorio, cada tabla e índice se representa mediante uno o más archivos nombrados con su relfilenode, divididos en segmentos de 1 GB y organizados internamente en páginas de 8 kB. A eso se suman los archivos de pg_wal (segmentos de 16 MB con el registro de todas las transacciones) y los directorios de metadatos globales como pg_global y pg_xact. El nombre lógico sgbd_lab y el nombre lógico cuentas sólo existen en el catálogo: en disco lo que hay son números.

9. Plan de ejecución

Con cinco registros el optimizador siempre elige recorrer la tabla completa, porque leer cinco filas cuesta menos que abrir un índice. Para que la diferencia fuera observable se creó una tabla auxiliar movimientos con aproximadamente un millón de filas relacionadas con cuentas, y se midió la misma consulta con los índices deshabilitados y habilitados.

9.1. Consulta utilizada

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT c.nro_cuenta, c.titular, count(*) AS movs, sum(m.importe) AS total
FROM cuentas c
JOIN movimientos m ON m.cuenta_id = c.id
WHERE c.titular LIKE '%Zurbriggen%'
GROUP BY 1, 2;
```

9.2. Plan antes de utilizar el índice

```
SET enable_indexscan = off;
SET enable_bitmapscan = off;
EXPLAIN (ANALYZE, BUFFERS) ... -- misma consulta
```

Resultado: recorrido secuencial (Seq Scan) sobre movimientos y Hash Join, con lectura de la totalidad de los bloques de la tabla. Tiempo de ejecución aproximado: **1100 ms**.

9.3. Plan después de crear el índice

```
RESET enable_indexscan;
RESET enable_bitmapscan;
EXPLAIN (ANALYZE, BUFFERS) ... -- misma consulta
```

Resultado: acceso por índice (Index Scan / Bitmap Index Scan sobre idx_mov_cuenta_id) y Nested Loop, leyendo únicamente los bloques de las filas que corresponden a la cuenta buscada. Tiempo de ejecución aproximado: **55 ms**.

Escenario	Método de acceso	Tiempo aproximado
Sin índice (índices deshabilitados)	Seq Scan + Hash Join	≈ 1100 ms
Con índice disponible	Index/Bitmap Scan + Nested Loop	≈ 55 ms

9.4. Interpretación

La mejora de aproximadamente veinte veces se explica por la cantidad de bloques leídos, no por la velocidad del disco: el recorrido secuencial obliga a traer a memoria todas las páginas de la tabla y descartar la enorme mayoría de las filas, mientras que el acceso por índice localiza directamente los punteros a las filas relevantes. La sección BUFFERS del plan es la que hace visible esa diferencia.

9.5. ¿La existencia de un índice obliga al optimizador a utilizarlo?

No. PostgreSQL utiliza un optimizador **basado en costos**: enumera los planes posibles, estima el costo de cada uno con las estadísticas que mantiene el autovacuum/ANALYZE y ejecuta el más barato, que no siempre es el que usa el índice. Nuestra propia evidencia lo confirma en dos sentidos: sobre cuentas (cinco filas) el motor prefiere el recorrido secuencial aunque exista el índice único sobre nro_cuenta, y sobre movimientos (un millón de filas) elige el índice apenas se lo habilita. El índice es una alternativa que se le ofrece al planificador, no una instrucción que deba obedecer.

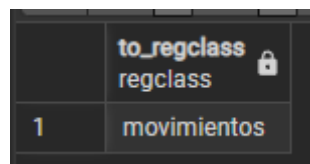
10. Persistencia

1. Se insertó y confirmó información nueva: además de los registros de cuentas, se creó y cargó la tabla movimientos con aproximadamente un millón de filas.

```
CREATE TABLE movimientos (  
  id          bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
  cuenta_id  integer      NOT NULL,  
  fecha      date         NOT NULL,  
  importe    numeric(12,2) NOT NULL,  
  tipo       varchar(10)  NOT NULL  
);
```

2. Se verificó la existencia del objeto antes de detener el motor:

```
SELECT to_regclass('public.movimientos');
```



El catálogo confirma que la relación existe.

3. Se detuvo el servicio del motor:

```
sudo systemctl stop postgresql  
sudo systemctl status postgresql
```

```

facubas@UbuntuServer:~$ sudo systemctl stop postgresql
facubas@UbuntuServer:~$ sudo systemctl status postgresql
* postgresql.service - PostgreSQL RDBMS
   Loaded: loaded (/usr/lib/systemd/system/postgresql.service; enabled; preset: enabled)
   Active: inactive (dead) since Sun 2026-08-23 22:28:16 UTC; 3s ago
     Duration: 32.371s
  Invocation: bae4f0a37e5a4aaebb775a314700bfd5
    Process: 9457 ExecStart=/bin/true (code=exited, status=0/SUCCESS)
   Main PID: 9457 (code=exited, status=0/SUCCESS)
    Mem peak: 1.7M
       CPU: 9ms

Aug 23 22:27:44 UbuntuServer systemd[1]: Starting postgresql.service - PostgreSQL RDBMS...
Aug 23 22:27:44 UbuntuServer systemd[1]: Finished postgresql.service - PostgreSQL RDBMS.
Aug 23 22:28:16 UbuntuServer systemd[1]: postgresql.service: Deactivated successfully.
Aug 23 22:28:16 UbuntuServer systemd[1]: Stopped postgresql.service - PostgreSQL RDBMS.
facubas@UbuntuServer:~$

```

Servicio detenido: inactive (dead).

4. Se volvió a iniciar el servicio:

```

sudo systemctl start postgresql
sudo systemctl status postgresql

```

```

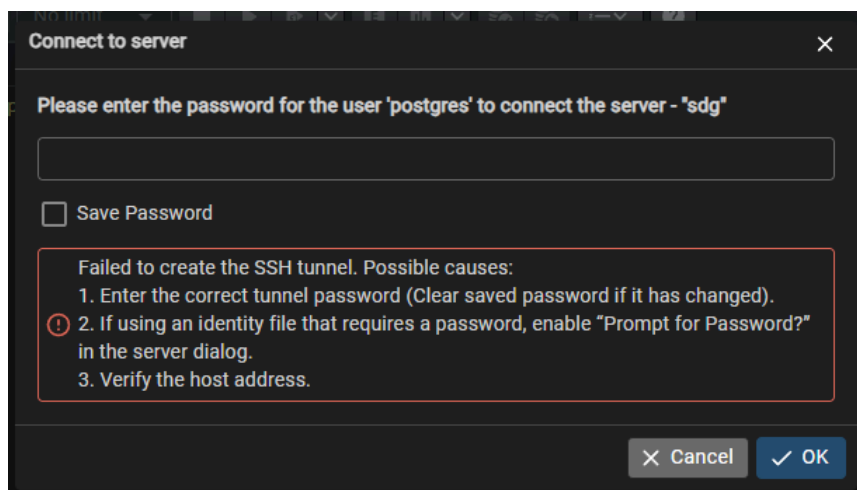
facubas@UbuntuServer:~$ sudo systemctl start postgresql
facubas@UbuntuServer:~$ sudo systemctl status postgresql
• postgresql.service - PostgreSQL RDBMS
   Loaded: loaded (/usr/lib/systemd/system/postgresql.service; enabled; preset: enabled)
   Active: active (exited) since Sun 2026-08-23 22:29:37 UTC; 13s ago
  Invocation: 7c4c69c708cd43ff922994879ac47077
    Process: 9592 ExecStart=/bin/true (code=exited, status=0/SUCCESS)
   Main PID: 9592 (code=exited, status=0/SUCCESS)
    Mem peak: 2M
       CPU: 8ms

Aug 23 22:29:36 UbuntuServer systemd[1]: Starting postgresql.service - PostgreSQL RDBMS...
Aug 23 22:29:37 UbuntuServer systemd[1]: Finished postgresql.service - PostgreSQL RDBMS.
facubas@UbuntuServer:~$

```

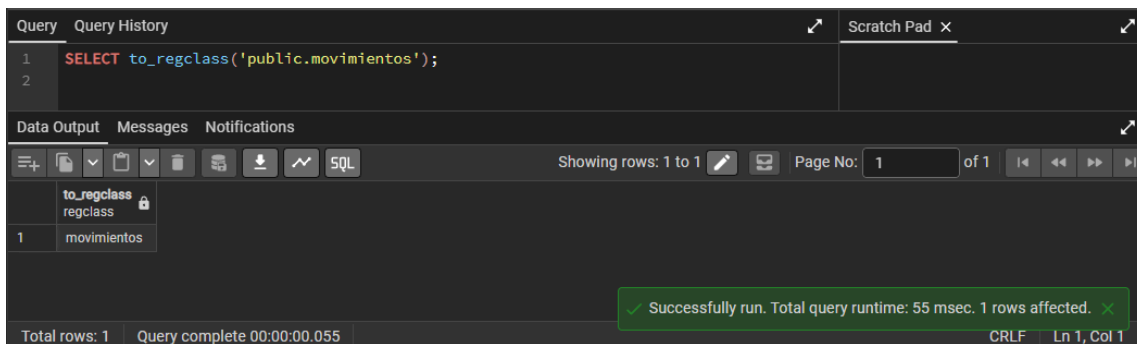
Servicio nuevamente activo tras el reinicio.

5. Se restableció la conexión desde pgAdmin 4 con el usuario postgres.



Reconexión al servidor luego del reinicio del motor.

6. Se consultó nuevamente la información y los objetos y datos seguían disponibles:



Tras detener e iniciar el motor, la relación movimientos sigue existiendo y la consulta se ejecuta correctamente.

¿Qué evidencia permite afirmar que la información es persistente?

La evidencia es que, después de un ciclo completo de detención e inicio del servicio —es decir, después de que toda la memoria del proceso servidor y el contenido de `shared_buffers` se perdieran—, tanto la definición de los objetos como sus datos continúan disponibles y consultables. Eso sólo es posible porque el motor escribe en disco y no depende de la memoria volátil.

El mecanismo que lo garantiza es el **WAL (Write-Ahead Log)**: antes de modificar una página de datos, PostgreSQL registra la modificación en el log transaccional y lo sincroniza a disco. Al confirmarse una transacción basta con que su registro WAL esté persistido; las páginas de datos se escriben después, en el checkpoint. Si el motor se detiene de forma abrupta, al reiniciar reproduce el WAL desde el último checkpoint y reconstruye el estado confirmado. Esto es lo que da soporte a la **D** de durabilidad en las propiedades ACID.

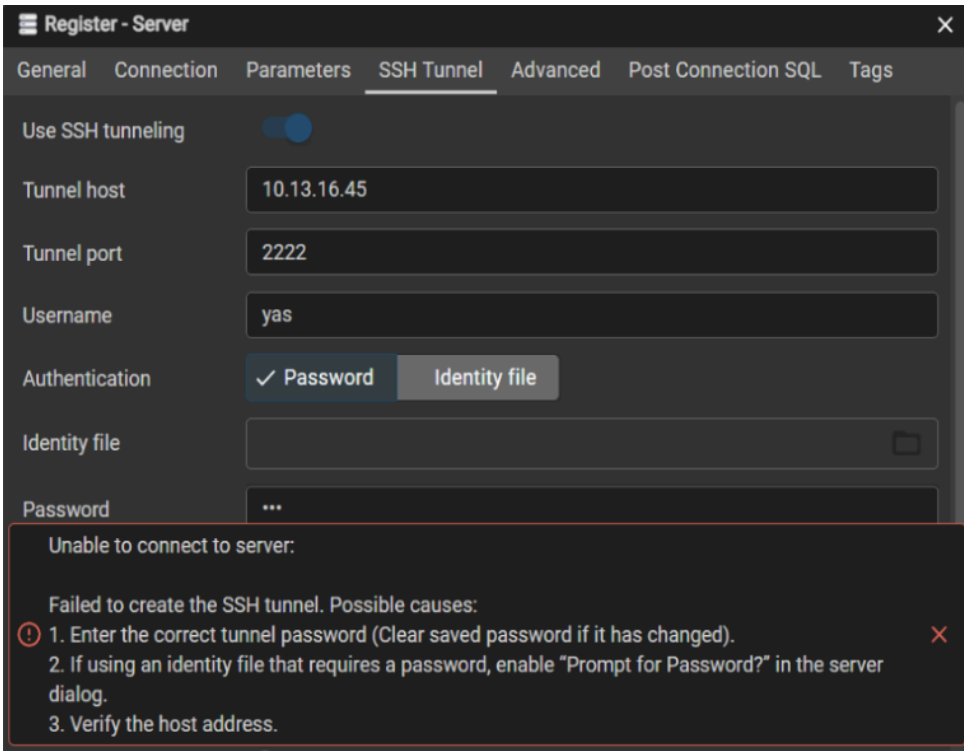
11. Mapa de arquitectura real

Correspondencia entre los conceptos teóricos de la Unidad 1 y los componentes concretos observados en la instancia instalada:

Concepto estudiado	Implementación encontrada
Instancia / servicio	postgresql@18-main.service (clúster 18-main), gestionado por la unidad envoltorio postgresql.service
Proceso principal	postgres (postmaster), PID 9850, usuario postgres
Base de datos	sgbd_lab
Esquema	public
Memoria / caché	shared_buffers (más work_mem y caché del sistema operativo)
Datos	PGDATA = /var/lib/postgresql/18/main (subdirectorío base/<OID>)
Índices	cuentas_pkey, cuentas_nro_cuenta_key, idx_mov_cuenta_id, idx_cuentas_titular_trgm
Log transaccional	WAL — /var/lib/postgresql/18/main/pg_wal
Metadatos	pg_catalog e information_schema
Configuración	postgresql.conf, pg_hba.conf, pg_ident.conf en /etc/postgresql/18/main/
Puerto	5432
Cliente CLI	psql 18.4

12. Incidente o decisión técnica

Incidente 1 · Falla del túnel SSH desde pgAdmin 4



Error obtenido al registrar el servidor mediante túnel SSH en pgAdmin 4.

Etapas	Detalle
Problema	Al registrar el servidor PostgreSQL en pgAdmin 4 usando un túnel SSH, la conexión falló con el mensaje "Failed to create the SSH tunnel. Possible causes: ... Verify the host address".
Diagnóstico	Los integrantes estaban conectados desde redes locales distintas. La dirección del servidor (10.13.16.45) es una IP privada, alcanzable únicamente dentro de la LAN del anfitrión de la VM, por lo que el intento de establecer el túnel nunca llegaba al destino.
Acción realizada	Se instaló y configuró Tailscale en los equipos de todos los integrantes y en la VM, creando una red virtual privada tipo mesh. Cada equipo recibió una dirección alcanzable dentro de esa red virtual y se ajustaron en pgAdmin 4 los parámetros de host, puerto y usuario del túnel SSH.
Resultado	El túnel SSH se estableció correctamente y todos los integrantes pudieron conectarse de manera concurrente a la misma instancia de PostgreSQL, trabajando sobre la base sgbd_lab en simultáneo.

Incidente 2 · Identificador creado con mayúscula inicial

Etapas	Detalle
Problema	La tabla se creó desde el asistente gráfico como "Cuentas" y todas las consultas escritas como SELECT ... FROM cuentas devolvían error de relación inexistente.
Diagnóstico	PostgreSQL pasa a minúsculas los identificadores no entrecomillados, pero conserva las mayúsculas cuando el identificador se crea entre comillas dobles, como hace pgAdmin. En consecuencia, "Cuentas" y cuentas son dos nombres distintos.
Acción realizada	Se renombró el objeto a minúsculas con ALTER TABLE "Cuentas" RENAME TO cuentas y se completaron la identidad de la clave primaria y la restricción UNIQUE sobre nro_cuenta.
Resultado	La tabla quedó accesible sin comillas desde cualquier cliente y se adoptó como criterio del grupo nombrar todos los objetos en minúsculas y con guión bajo.

Decisión técnica · Repositorio de instalación

Se decidió instalar el motor desde el repositorio de la distribución en lugar de dar de alta el repositorio PGDG del proyecto. La razón es que la versión candidata ofrecida por Ubuntu ya era 18.4, la misma que indicaba la documentación oficial, de modo que agregar un origen de paquetes externo no aportaba una versión más nueva y sí sumaba puntos de mantenimiento (clave GPG, archivo de sources, posibles conflictos de dependencias). Si el requerimiento hubiera sido una versión mayor a la empaquetada por la distribución, la decisión habría sido la inversa.

13. Uso de IA

¿Para qué se utilizó?

- Como asistente de consulta técnica para orientar el paso a paso de las actividades y resolver dudas de sintaxis de comandos de Linux y de sentencias SQL de PostgreSQL.
- Para interpretar mensajes de error, en particular el fallo del túnel SSH de pgAdmin 4, y evaluar alternativas de conectividad entre integrantes ubicados en redes distintas.
- Para generar el volumen de datos de prueba de la tabla movimientos y así poder observar una diferencia medible entre una consulta con índice y otra sin índice.

¿Qué información aportó?

- Explicaciones sobre la arquitectura interna del motor: relación entre instancia, procesos, memoria compartida y archivos en disco.
- Ejemplos de comandos de administración por CLI y de consultas al catálogo del sistema.
- Rutas convencionales de los archivos de configuración, datos y logs en el empaquetado de Debian/Ubuntu.
- Alternativas de solución ante fallos de conectividad, entre ellas el uso de una red virtual tipo mesh.

¿Cómo fue validada?

Ninguna afirmación se incorporó a la bitácora sin comprobación previa. Cada comando sugerido se ejecutó en la máquina virtual o en pgAdmin 4 y se registró su salida real, que es la que se adjunta como evidencia en este informe. Las rutas y los nombres de servicios se verificaron consultando al propio motor (`SHOW data_directory`, `SHOW config_file`) y al sistema operativo (`systemctl status, ps`), en lugar de darlas por válidas. Los conceptos se contrastaron con la documentación oficial de PostgreSQL. Cuando una sugerencia no coincidió con lo observado, prevaleció la evidencia del entorno.

14. Conclusión

¿Qué concepto estudiado en la Unidad 1 comprendieron mejor después de instalar y explorar el motor asignado?

El concepto que mejor comprendimos fue el de niveles de abstracción e independencia física de los datos. En la teoría entendíamos la separación entre el nivel lógico y el nivel físico, pero recién en el laboratorio la vimos funcionando: creamos una base, un esquema y una tabla usando SQL estándar, sin decidir en ningún momento en qué archivo ni en qué bloque del disco se guardaría cada fila.

Al recorrer los archivos comprobamos que ese nombre (`sgbd_lab`, cuentas) no existe en el disco: hay directorios y archivos identificados por OID, páginas de 8 kB, segmentos de WAL y un usuario postgres ejecutando un proceso servidor. El puente entre ambos mundos es el catálogo del sistema, que además es consultable. La prueba de persistencia y la comparación de planes de ejecución completaron la idea: el motor decide por su cuenta cómo acceder a los datos y cómo garantizar que sobrevivan a un reinicio, y esas decisiones quedan ocultas detrás de una interfaz declarativa estable.